

Manual

Deepfake Detection System — Command Center Dashboard

User Manual

Detection of Social Media Deepfake Contents Using Deep
Learning Algorithm Olamijulo Israel D (CYS/22/9071) — Federal
University of Technology Akure

Table of Contents

1. [Project Overview](#)
 2. [System Requirements](#)
 3. [Installation](#)
 4. [Launching the Dashboard](#)
 5. [Dashboard Pages](#)
 - [HOME](#)
 - [ANALYZE](#)
 - [TRAIN](#)
 - [EVALUATE](#)
 - [DATASETS](#)
 - [EXPERIMENTS](#)
 - [SYSTEM](#)
 6. [Step-by-Step: Testing Images](#)
 7. [Step-by-Step: Testing Videos](#)
 8. [Step-by-Step: Testing YouTube URLs](#)
 9. [Understanding Results](#)
 10. [Model Selection Guide](#)
 11. [Troubleshooting](#)
 12. [Project Structure](#)
-

1. Project Overview

This system is a deepfake detection tool built for social media content analysis. It uses two deep learning models to classify images and videos as **REAL** or **FAKE**:

- **XceptionNet** — Primary model, 22.9M parameters, 99.78% accuracy
- **EfficientNet-B0** — Lightweight model, 4.01M parameters, 99.23% accuracy

The dashboard is built with Streamlit and runs as a local web application. It features a dark cybersecurity/hacker terminal aesthetic.

What it can do: - Analyze uploaded images (JPG, PNG, BMP, WebP) - Analyze uploaded videos (MP4, AVI, MOV, MKV) - Download and analyze YouTube videos directly by URL - Batch-analyze multiple files at once - Display model evaluation metrics and confusion matrices - Compare model performance side by side

2. System Requirements

Component	Requirement
Python	3.10+
RAM	8 GB minimum, 16 GB recommended
Disk	~200 MB (dependencies + checkpoints)
GPU	Optional (CPU works, GPU is faster)
OS	Windows, macOS, Linux

3. Installation

Quick Setup (Recommended)

```
# 1. Clone the repository
git clone https://github.com/Nico3783/deepfake-social-media-detector.git
cd deepfake-social-media-detector

# 2. Create a virtual environment
python -m venv venv
```

```
# 3. Activate the virtual environment
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate

# 4. Install dependencies
pip install -r requirements.txt

# 5. (Optional) Install YouTube support
pip install yt-dlp
```

Minimal Setup (If requirements.txt fails)

```
pip install streamlit torch torchvision opencv-python pillow
      pandas plotly scikit-learn numpy
```

Verify Checkpoints

Trained model checkpoints must exist in `outputs/checkpoints/`: -
`xceptionnet_best.pth` (~66 MB) - `efficientnet-b0_best.pth` (~16 MB)

If missing, download them from the GitHub releases or retrain.

4. Launching the Dashboard

```
# From the project root directory:
streamlit run app.py
```

The dashboard opens at **`http://localhost:8501`** in your default browser.

Common Launch Issues

Problem	Fix
streamlit: command not found	Activate your virtual environment first
ModuleNotFoundError	Run <code>pip install -r requirements.txt</code> again
Port already in use	Use <code>streamlit run app.py --server.port 8502</code>

Problem	Fix
Black screen	Hard refresh with Ctrl+Shift+R

5. Dashboard Pages

5.1 HOME

The landing page showing: - **System architecture diagram** — visual flow from Input → Preprocessing → CNN Model → Aggregator → REAL/FAKE - **Model descriptions** — XceptionNet and EfficientNet-B0 specifications - **Recent activity** — table of analyses performed in the current session - **Thesis information** — project metadata (student name, department, institution)

5.2 ANALYZE

The main analysis page with three tabs:

VIDEO ANALYSIS — Upload a video file or paste a YouTube URL - YouTube URL input with “DOWNLOAD & ANALYZE” button - File upload (MP4, AVI, MOV, MKV) - Displays video preview, file info, and model/threshold settings - Shows fake/real probability bar after analysis

IMAGE ANALYSIS — Upload a single image - Supported: JPG, JPEG, PNG, BMP, WebP - Shows image preview and file metadata - Outputs prediction with confidence and face detection status

BATCH PROCESS — Upload multiple files at once - Mix of images and videos in one batch - Results displayed in a table with per-file predictions - Supports drag-and-drop of multiple files

Analysis Settings (expandable panel): - **Model**: Choose between XceptionNet and EfficientNet-B0 - **Confidence Threshold**: Slider from 0.0 to 1.0 (default: 0.5) - **Aggregation** (video only): mean, majority, or `confidence_weighted`

5.3 TRAIN

Model training configuration page (simulated in dashboard): - Configure hyperparameters: learning rate, batch size, epochs, optimizer, loss function - Select dataset: FaceForensics++, Celeb-DF, or Both - View simulated training progress with live loss/accuracy charts - Lists saved checkpoints in outputs/checkpoints/

Note: Actual training was performed in Google Colab. This page is a simulation for demonstration purposes.

5.4 EVALUATE

Model performance analysis with four tabs: - **METRICS** — Accuracy, Precision, Recall, F1 Score, AUROC - **CONFUSION MATRIX** — True Positives, False Positives, etc. - **ROC CURVE** — Receiver Operating Characteristic visualization - **REPORTS** — Generate JSON, Markdown, or LaTeX reports

5.5 DATASETS

Dataset information and management: - **DATASET INFO** — Details about FaceForensics++ and Celeb-DF v2 - **EXPLORE DATA** — View metadata CSV files - **SPLIT MANAGER** — Configure train/val/test splits (70/15/15)

5.6 EXPERIMENTS

Experiment tracking and comparison: - **COMPARE MODELS** — Side-by-side XceptionNet vs EfficientNet-B0 table - **EXPERIMENT LOG** — Browse saved experiment JSON files - **ANALYSIS HISTORY** — View and export session analysis history as CSV

5.7 SYSTEM

System health and diagnostics: - **HEALTH** — System status, GPU/CPU, VRAM, Python version, component check - **CONFIGURATION** — View YAML config files - **DEPENDENCIES** — List all installed packages

6. Step-by-Step: Testing Images

1. Open the dashboard at <http://localhost:8501>

2. Click **ANALYZE** in the sidebar
 3. Click the **IMAGE ANALYSIS** tab
 4. Expand **ANALYSIS SETTINGS** and select your model (XceptionNet recommended)
 5. Adjust the **Confidence Threshold** if needed (0.5 = balanced)
 6. Click **Browse files** or drag-and-drop an image
 7. The image preview and metadata appear
 8. Click **ANALYZE IMAGE**
 9. Results appear:
 - **REAL** (green border) or **FAKE** (red border)
 - Confidence percentage
 - Fake/Real probability scores
 - Whether a face was detected
-

7. Step-by-Step: Testing Videos

1. Click **ANALYZE** in the sidebar
 2. Click the **VIDEO ANALYSIS** tab
 3. Configure settings:
 - Select model (XceptionNet recommended)
 - Set confidence threshold (0.5 default)
 - Choose aggregation method (mean recommended)
 4. Upload a video file (MP4, AVI, MOV, MKV)
 5. The video preview and file info appear
 6. Click **RUN ANALYSIS**
 7. The system extracts frames, runs inference on each, and aggregates results
 8. Results appear:
 - **REAL** or **FAKE** prediction
 - Confidence score
 - Number of frames analyzed
 - Fake/Real probability bar
 9. Click **DETAILED REPORT** expander to see full JSON output
-

8. Step-by-Step: Testing YouTube URLs

1. Click **ANALYZE** in the sidebar
2. Click the **VIDEO ANALYSIS** tab
3. Paste a YouTube URL into the **YouTube URL** input field
 - Example: <https://www.youtube.com/watch?v=dQw4w9WgXcQ>
4. Click **DOWNLOAD & ANALYZE**

5. The system:

- Downloads the video (best quality $\leq 720p$) via yt-dlp
- Saves it temporarily to outputs/temp/youtube/
- Runs the same video analysis pipeline
- Cleans up the temporary file

6. Results appear with the same format as uploaded video analysis

Note: Requires yt-dlp to be installed. If not, you'll see a warning with installation instructions.

9. Understanding Results

Prediction Output

Field	Description
Prediction	REAL or FAKE
Confidence	How certain the model is (0-100%)
Fake Probability	Raw score for fake class (0.0-1.0)
Real Probability	Raw score for real class (0.0-1.0)
Frames Analyzed	Number of video frames processed (images: 1)
Face Detected	Whether a face was found in the image

Interpreting Scores

- **Fake Probability** > 0.5 → Model leans toward FAKE
- **Fake Probability** < 0.5 → Model leans toward REAL
- **Confidence** > 80% → High certainty prediction
- **Confidence** 50-80% → Moderate certainty, may need manual review
- **Confidence** < 50% → Low certainty, treat with caution

Aggregation Methods (Video)

- **mean** — Averages all frame probabilities (recommended, most balanced)
 - **majority** — Majority vote of frame predictions
 - **confidence_weighted** — Weights each frame by its confidence score
-

10. Model Selection Guide

Scenario	Recommended Model
Maximum accuracy needed	XceptionNet
Limited compute / faster results	EfficientNet-B0
Mobile/edge deployment	EfficientNet-B0
Research/benchmarking	Use both and compare

Property	XceptionNet	EfficientNet-B0
Parameters	22.9M	4.01M
Input Size	299×299	224×224
Accuracy	99.78%	99.23%
Inference Speed	~8 ms/frame	~4 ms/frame
Checkpoint Size	~66 MB	~16 MB

11. Troubleshooting

Problem	Solution
No trained model checkpoint found	Ensure .pth files exist in outputs/checkpoints/
Video analysis is slow	Use EfficientNet-B0 or reduce video length
YouTube download fails	Check URL, ensure video is public, update yt-dlp (pip install -U yt-dlp)
Dashboard shows “CPU” in SYSTEM	Normal if no CUDA GPU is available
ModuleNotFoundError: torch	Run pip install torch torchvision
Streamlit crashes on large videos	Try shorter videos or lower resolution
Port 8501 already in use	Kill existing process or use --server.port 8502

12. Project Structure

```
deepfake-social-media-detector/
├─ app.py                    # Streamlit dashboard (main
entry point)
├─ requirements.txt          # Python dependencies
├─ manual.md                 # This file
├─ LICENSE
├─ README.md
├─ .gitignore
├─ .streamlit/
│   └─ config.toml          # Streamlit configuration
(dark theme, port 8501)
├─ configs/
│   ├── training.yaml        # Training configuration
│   ├── xception.yaml        # XceptionNet config
│   ├── efficientnet.yaml    # EfficientNet config
│   ├── dataset.yaml         # Dataset config
│   └─ inference.yaml        # Inference config
├─ src/
│   ├── models/
│   │   └─ model_factory.py  # Model loading (load_model)
│   ├── inference/
│   │   ├── predict_image.py # ImagePredictor class
│   │   └─ predict_video.py  # VideoPredictor class
│   └─ data/
│       └─ organize.py        # Dataset organization
├─ outputs/
│   ├── checkpoints/
│   │   ├── xceptionnet_best.pth # Trained XceptionNet (66 MB)
│   │   └─ efficientnet-b0_best.pth # Trained EfficientNet-B0 (16
MB)
│   ├── experiments/         # Experiment JSON logs
│   ├── reports/             # Generated reports
│   └─ temp/                 # Temporary files (YouTube
downloads, gitignored)
├─ datasets/
│   └─ metadata/             # Dataset CSV metadata
└─ thesis/
    └─ thesis.docx           # Final thesis document
```

Quick Reference Card

Action	Command
Launch dashboard	streamlit run app.py
Install dependencies	pip install -r requirements.txt

Action	Command
Install YouTube support	<code>pip install yt-dlp</code>
Custom port	<code>streamlit run app.py -- server.port 8502</code>
Export analysis history	<code>ANALYZE → EXPERIMENTS → ANALYSIS HISTORY → EXPORT HISTORY</code>

Manual version 1.0 — July 2026